# Azure Basics for the NYC TLC Ingestion Project (Week 1)

> **Goal of this doc:** Teach you *just enough* Azure — with the "why" behind every step — so you can build the ADF ingestion pipeline yourself AND explain it confidently in an interview. No extra fluff, no features you won't use in this project.

---

## Table of Contents


1. What is "the Cloud" and why does this project use Azure
2. Core Azure Building Blocks (know these 5 words)
3. Setting Up Your Free Azure Account
4. Storage: What is ADLS Gen2 and why not just "Blob Storage"
5. Creating Your ADLS Gen2 Account (step-by-step)
6. Understanding Access: Keys, SAS, and Managed Identity
7. What is Azure Data Factory (ADF) and why do we need it
8. The 4 ADF Building Blocks You Must Know
9. Building the Pipeline: Step-by-Step with Examples
10. Parameterization — Making It Loop Over Months/Years
11. Running, Monitoring, and Debugging
12. Interview Cheat Sheet
13. Glossary (quick lookup)


---

## 1. What is "the Cloud" and why does this project use Azure

Forget the marketing language for a second. Here's the plain version:

- A **cloud provider** (Azure, AWS, GCP) is basically **someone else's computers and hard drives**, that you rent by the hour/GB, instead of buying your own servers.
- Instead of buying a physical hard disk to store 1TB of taxi data, you rent storage space on Microsoft's data centers. Instead of buying a powerful server to process that data, you rent "compute power" for the hours you actually use it, then shut it off.

**Why this matters for your project specifically:** NYC TLC data is huge (~1TB across years). You cannot download, store, and process that on a laptop. Cloud lets you:

- Store huge files cheaply (ADLS Gen2)
- Move files around automatically on a schedule (ADF)

- Process huge files in parallel across many machines (Databricks/Spark)
- Pay only for what you use, and turn it off when done

**Why Azure specifically (not AWS/GCP)?** No deep technical reason for a portfolio project — Azure is chosen here because it pairs naturally with Databricks (Databricks was co-founded to work great on Azure, called "Azure Databricks") and because ADF is a strong, resume-worthy orchestration tool. In an interview, you can honestly say: *"I picked Azure because ADF + ADLS Gen2 + Azure Databricks is one of the most common enterprise data engineering stacks, and I wanted hands-on experience with tools companies actually use."*

---

## 2. Core Azure Building Blocks

Before touching anything, know these 5 words. Every Azure resource you create lives inside this hierarchy:

```
Azure Account (your login)
   └── Subscription (billing boundary — "free tier" is a subscription)
         └── Resource Group (a folder/box that groups related resources)
               └── Resources (the actual things: storage account, ADF, etc.)
```

| Term | What it actually is | Analogy |
|---|---|---|
| **Subscription** | Your billing container. Free tier = one subscription with $200 credit for 30 days + some always-free services | Your bank account |
| **Resource Group (RG)** | A logical folder that holds related resources so you can manage/delete them together | A project folder on your laptop |
| **Region** | The physical location of Microsoft's data center (e.g., "Central India", "East US") | Which warehouse your stuff is stored in |
| **Resource** | An actual service instance — a storage account, a Data Factory, a VM | A file inside the folder |

**Why Resource Groups matter for you:** At the end of this project, you delete the *whole Resource Group* and everything inside it (storage, ADF, etc.) gets deleted in one click. This is how you avoid getting billed after you're done experimenting — **this is your #1 cost-safety habit.**

> **Interview tip:** If asked "how did you structure your Azure resources," say: *"I created a single resource group for the project so I could manage cost and clean-up centrally — all resources for the pipeline (storage account, Data Factory) lived in one RG."*

---

## 3. Setting Up Your Free Azure Account

1. Go to https://azure.microsoft.com/free — sign up with a Microsoft account (or create one).

2. You'll need a credit/debit card for identity verification — **you will not be auto-charged** after the free credit unless you manually upgrade.
3. You get:
    - $200 credit for 30 days (use for VMs, Databricks compute, etc.)
    - Some services free for 12 months
    - Some services **always free** up to a limit
4. Once logged in, you land on the **Azure Portal** (portal.azure.com) — this is the web dashboard (GUI) where you'll click around and create resources.

**First thing to do:** Create your Resource Group.

- Search bar (top) → type "Resource groups" → **+ Create**
- Name: `rg-nyctlc-project`
- Region: pick the one closest to you (e.g., `Central India` if you're in Mumbai) — **use the SAME region for every resource in this project.** Different regions = extra data transfer cost + latency, and it looks sloppy in an architecture review.

  **Why same region matters (interview-worthy point):** Data transfer between services in the *same* region is free and fast. Cross-region transfer costs money and adds latency. Always design with a single region unless you have a specific reason (e.g., disaster recovery, data residency laws).

---

# 4. Storage: ADLS Gen2

## What's the difference between "Blob Storage" and "ADLS Gen2"?

This trips up almost everyone, so here's the plain-English version:

- **Azure Blob Storage** = a giant flat bucket where you dump files. Technically there are no real "folders" — folder-looking paths are just part of the filename (e.g., a file named `raw/2023/01/data.parquet` *looks* like it's in folders, but it's really one flat namespace pretending to have folders).
- **ADLS Gen2 (Azure Data Lake Storage Gen2)** = Blob Storage **+ a real folder system** (called a "hierarchical namespace"), **+** fine-grained security per folder, **+** it's built/optimized for big data engines like Spark and Databricks to read efficiently.

**Why we use ADLS Gen2 for this project and not plain Blob Storage:**

1. Real folders → you can organize `raw/`, `bronze/`, `silver/`, `gold/` zones cleanly (this maps directly to your medallion architecture in Weeks 2-3).
2. Renaming/moving a folder is one fast operation, not thousands of file renames (matters a lot at 1TB scale).
3. Databricks and Spark are built to read ADLS Gen2 natively and efficiently.
4. You can set security permissions per folder (e.g., "raw" folder read-only for some users) —

a real enterprise pattern.

**Interview line:** *"I used ADLS Gen2 instead of plain Blob Storage because it provides a hierarchical namespace, which lets Spark and Databricks perform directory-level operations efficiently and lets me organize data into medallion zones — raw, bronze, silver, gold — as real folder structures rather than simulated prefixes."*

### Containers vs Folders

- A **Storage Account** is the top-level resource (like the whole hard disk).
- Inside it, you create **Containers** (like top-level drives, e.g., `C:` and `D:`).
- Inside a container, you create **folders** (only possible because Hierarchical Namespace is ON — that's the "Gen2" part).

**Recommended structure for this project:**

```
storageaccount: stnyctlcproject
 └── container: datalake
       ├── raw/              ← Week 1: untouched files landed by ADF, exactly
as downloaded
       │    └── yellow_tripdata/
       │          └── 2023/
       │                └── 01/
       │                      └── yellow_tripdata_2023-01.parquet
       ├── bronze/           ← Week 2: raw data loaded into Delta tables, as-is
       ├── silver/           ← Week 2-3: cleaned, deduped, typed data
       └── gold/             ← Week 3: aggregated, business-ready tables
```

---

## 5. Creating Your ADLS Gen2 Account

Step-by-step in the Azure Portal:

1. Search bar → "Storage accounts" → **+ Create**
2. **Basics tab:**
   - Resource group: `rg-nyctlc-project` (the one you made earlier)
   - Storage account name: must be globally unique, lowercase, no spaces (e.g., `stnyctlcproj001`)
   - Region: same as your resource group
   - Performance: **Standard** (Premium is overkill/expensive for this project)
   - Redundancy: **LRS (Locally Redundant Storage)** — cheapest option, keeps 3 copies within one data center. (GRS/ZRS cost more and replicate across regions — not needed for a portfolio project)
3. **Advanced tab (⚠ the most important step):**
   - Find **"Enable hierarchical namespace"** → turn this **ON**.
   - This single checkbox is what turns plain Blob Storage into ADLS Gen2. If you forget this, you don't have a data lake — you have a flat bucket.

4. Review + Create → wait ~1 minute for deployment.
5. Once created, go into the storage account → **Containers** (left menu) → **+ Container** → name it `datalake`.
6. Inside the container, manually create your `raw` folder (you can do this via the portal's "Add Directory" button, or let ADF create it automatically when the pipeline runs — either is fine).

> **Cost note:** Standard LRS storage costs roughly $0.02/GB/month (region-dependent) at the "Hot" access tier. For a learning project, this is trivially cheap even at 1TB (~$20/month if you kept 1TB sitting there for a full month — but you'll likely process and reduce/aggregate the data quickly, and can delete raw files after bronze is loaded).

---

## 6. Understanding Access

You need a way for ADF to be *allowed* to read/write your storage account. There are 3 common methods — know all 3 conceptually, you'll likely use #1 or #3.

| Method | What it is | When to use |
|---|---|---|
| **Account Key** | A long secret password for the whole storage account (full access) | Quick learning/dev projects — simplest, but least secure (whoever has the key has full access) |
| **SAS Token (Shared Access Signature)** | A temporary, scoped key — "read-only access to this container for 7 days" | When you want limited/expiring access |
| **Managed Identity** | Azure automatically gives your ADF service its own "identity" that you grant permissions to (via RBAC roles) — no secrets/passwords involved at all | **Production best practice** — no keys to leak, ever |

**For this project:** Start with **Account Key** to get moving fast (it's in Storage Account → "Access keys" in the portal). Once your pipeline works, **switch to Managed Identity** and mention this upgrade in your README/interview — it's a strong signal of security awareness.

> **Interview line:** *"I initially used account key auth to get the pipeline working quickly, then migrated to Managed Identity so ADF authenticates to ADLS Gen2 without any stored secrets — this follows the principle of least privilege and avoids credential leakage risk."*

To set this up later: Storage Account → **Access Control (IAM)** → **Add role assignment** → role = `Storage Blob Data Contributor` → assign to → your Data Factory's managed identity.

---

# 7. What is Azure Data Factory (ADF)

## The problem ADF solves

You need to download 12+ months × several years of NYC TLC parquet files and land them in ADLS Gen2, *without* manually clicking "download" hundreds of times, and ideally in a way that can repeat itself on a schedule.

You *could* write a Python script for this. So why use ADF instead?

- **ADF is a managed orchestration service** — it's built for exactly this: moving data from a source (NYC TLC's public URL) to a destination (ADLS Gen2), on a schedule, with retries, logging, and monitoring built in — with almost no code.
- It has a **visual, drag-and-drop pipeline builder** — this matters in real companies because non-engineers (analysts) can sometimes maintain simple pipelines too.
- It's **serverless** — you don't manage any servers; Microsoft runs the actual compute behind the scenes ("Integration Runtime").
- It has native **parameterization and looping**, which is exactly your requirement: "loop over months/years."

> **Interview line:** *"I used ADF instead of a plain Python script because it gives me managed orchestration — built-in retry logic, monitoring, alerting, and scheduling — without provisioning any infrastructure. It's also the industry-standard orchestration tool in Azure data engineering stacks, so it directly translates to what I'd use on the job."*

## ETL vs ELT — know this distinction

- **ETL** (Extract, Transform, Load): transform data *before* loading it into the destination.
- **ELT** (Extract, Load, Transform): load raw data first, transform *after*, inside the destination system.

**This project is ELT.** ADF's job in Week 1 is purely **E + L**: extract the parquet file from NYC's website, land it as-is into `raw/` in ADLS Gen2. **No transformation happens in ADF.** All transformation (cleaning, dedup, typing) happens later in Databricks (Bronze → Silver → Gold). This is the modern "ELT" pattern used in real data lakes, and you should say this explicitly in interviews — it shows you understand *why* the raw zone exists (it's your untouched, replayable source of truth).

---

# 8. The 4 ADF Building Blocks

Every ADF pipeline is made of these 4 concepts. Learn these words cold — this is 80% of the ADF interview vocabulary.

## 1. Linked Service — "the connection info"

Think of it as a saved connection string / credentials to a system. You'll create two:

- **Linked Service #1**: HTTP connection to NYC TLC's public data URL (source)
- **Linked Service #2**: Connection to your ADLS Gen2 storage account (destination), using the Account Key or Managed Identity from Section 6.

### 2. Dataset — "the shape of the data at a specific location"

A Dataset points to a *specific file or table*, using a Linked Service, and describes its format.

- **Dataset #1**: the source parquet file — built as a *parameterized* HTTP file (URL changes based on year/month — more in Section 10)
- **Dataset #2**: the destination file path in ADLS Gen2 — also parameterized (folder path changes based on year/month)

### 3. Activity — "a single step/task"

An Activity is one action inside a pipeline. You'll mainly use:

- **Copy Activity**: copies data from a source dataset to a sink (destination) dataset. This is your core "download and land the file" step.
- **ForEach Activity**: loops over a list of values (e.g., all 12 months) and runs an inner activity (the Copy Activity) once per item.

### 4. Pipeline — "the whole workflow"

A Pipeline is the container that holds and sequences your Activities. Your pipeline for Week 1 will basically be:

```
Pipeline: pl_ingest_nyc_tlc
   └── ForEach Activity (loops over a list of year-month combos)
         └── Copy Activity (downloads that month's parquet → lands in raw/)
```

### Integration Runtime (IR) — the "engine" behind the scenes

You don't design this much, but know what it is: the actual compute that executes your pipeline. For this project, the **default "AutoResolveIntegrationRuntime"** (Azure-managed, serverless) is enough — you'd only build a **Self-Hosted IR** if you needed ADF to reach data sitting inside a private on-prem network, which isn't your case here.

---

## 9. Building the Pipeline: Step-by-Step

### Step 1 — Create the Data Factory resource

- Portal → search "Data factories" → **+ Create**
- Resource group: `rg-nyctlc-project`
- Name: `adf-nyctlc-project`
- Region: same as before
- Version: **V2** (V1 is legacy, don't use it)

- Create → once deployed, click **"Launch Studio"** — this opens the actual visual pipeline builder (a separate web app: adf.azure.com)

## Step 2 — Create the HTTP Linked Service (source)

In ADF Studio → **Manage** (toolbox icon on left) → **Linked Services** → **+ New**

- Type: **HTTP**
- Base URL: `https://d37ci6vzurychx.cloudfloud.net/trip-data/` *(NYC TLC's actual public data host — verify the current base URL on the TLC page before using, providers occasionally change hosting)*
- Authentication type: **Anonymous** (it's a public dataset, no login needed)

## Step 3 — Create the ADLS Gen2 Linked Service (destination)

Same **Linked Services** area → **+ New**

- Type: **Azure Data Lake Storage Gen2**
- Authentication: Account Key (for now — pick your storage account from the dropdown, Azure auto-fills the key) — later upgrade to Managed Identity

## Step 4 — Create the Source Dataset (parameterized)

**Author** tab → **Datasets** → **+ New Dataset** → choose **HTTP** → format: **Binary** (parquet files are binary at this stage; you're not parsing them, just copying the raw bytes)

- Link it to your HTTP Linked Service
- Add **Parameters** to this dataset: `p_year` (string), `p_month` (string)
- Set the **Relative URL** field using dynamic content (the `fx` icon):

```
yellow_tripdata_@{dataset().p_year}-@{dataset().p_month}.parquet
```

  This means: if `p_year = 2023` and `p_month = 01`, the file requested is `yellow_tripdata_2023-01.parquet` — exactly NYC TLC's real naming pattern.

## Step 5 — Create the Sink Dataset (parameterized)

Another new Dataset → **Azure Data Lake Storage Gen2** → format: **Binary**

- Add the same parameters: `p_year`, `p_month`
- Set the **File path** using dynamic content:

```
datalake/raw/yellow_tripdata/@{dataset().p_year}/@{dataset().p_month}/
yellow_tripdata_@{dataset().p_year}-@{dataset().p_month}.parquet
```

  This automatically creates the folder structure you designed in Section 4 as files land.

## Step 6 — Build the Pipeline

**Author** → **Pipelines** → **+ New Pipeline** → name it `pl_ingest_nyc_tlc`

Drag a **Copy Data** activity onto the canvas:

- Source: your HTTP dataset → pass values for `p_year`/`p_month` (for now, hardcode `2023` / `01` to test)
- Sink: your ADLS Gen2 dataset → same parameter values
- Click **Debug** to test run it once. Check your storage account — you should see the file land in `raw/yellow_tripdata/2023/01/`.

Once this single-file copy works, move to Section 10 to make it loop.

---

# 10. Parameterization — Making It Loop Over Months/Years

This is the part your project description calls your **"first automation win."** Here's exactly how and why.

## Why hardcoding is bad

If you hardcode `2023`/`01` into the pipeline, you'd need to manually edit and re-run it 60+ times for 5 years of data. That's not automation — that's just clicking a button 60 times. Parameterization + looping means you define the pipeline **once**, and it works for any month/year you feed it.

## Step 1 — Add Pipeline Parameters (optional but clean)

At the pipeline level, you can define top-level parameters like `p_start_year`, `p_end_year` if you want full control. For simplicity, many people just hardcode the list of months to loop over directly in a variable (see Step 2).

## Step 2 — Add a Pipeline Variable holding your list of months

**Variables** tab (bottom of pipeline canvas) → **+ New**

- Name: `v_month_list`
- Type: Array
- Default value (example, one year):

```
["2023-01","2023-02","2023-03","2023-04","2023-05","2023-06",
 "2023-07","2023-08","2023-09","2023-10","2023-11","2023-12"]
```

## Step 3 — Add a ForEach Activity

Drag a **ForEach** activity onto the canvas (before or wrapping your Copy Activity).

- **Items** field → dynamic content:

```
@variables('v_month_list')
```

- Inside the ForEach's "Activities" canvas (double-click to go inside), place your **Copy Data** activity from Step 9.

### Step 4 — Split year and month from each loop item, and feed the datasets

Inside the ForEach, each loop iteration gives you `@item()` — a string like `"2023-01"`. You need to split it:

- For the source dataset's `p_year` parameter, use:

```
@split(item(), '-')[0]
```

- For `p_month`:

```
@split(item(), '-')[1]
```

This splits `"2023-01"` into `["2023", "01"]` and picks out each piece.

### Step 5 — Publish and Debug

Click **Publish All** (saves your pipeline), then **Debug** (or **Add Trigger → Trigger Now** for a real run). Watch the **Output** tab — you'll see each ForEach iteration run and either succeed (green check) or fail (red X) individually, which is great for troubleshooting one bad month without re-running everything.

> **Interview line (this is your "automation" talking point):** *"I parameterized both my source and sink datasets so the pipeline is completely generic — it doesn't know or care which month it's copying. I drive it with a ForEach activity iterating over an array of year-month strings, so adding 5 more years of data is a one-line change to an array, not a pipeline redesign. This is the same pattern used for production incremental-load pipelines."*

### Bonus: Scheduling (mention even if you don't run it live)

**Triggers → + New → Schedule** trigger → e.g., "run on the 5th of every month" (since TLC publishes each month's data a few days after month-end). You don't need to leave this running for a portfolio project (it'll rack up executions), but **building and explaining it** shows you understand production scheduling.

---

## 11. Running, Monitoring, and Debugging

- **Monitor tab** (left sidebar in ADF Studio) → shows every pipeline run, its duration, and status (Succeeded/Failed/InProgress).
- Click into a failed run → it tells you exactly which activity failed and the error message (e.g., "404 Not Found" usually means the URL pattern or year/month value was wrong).
- Common issues you'll likely hit (and what they teach you): | Error | Cause | Fix | |---|---|---| | 404 on HTTP source | Wrong file naming pattern or that month's file doesn't exist yet on TLC's site | Double check TLC's actual current URL format | | 403 on ADLS sink | Storage key/permissions not set correctly | Recheck Linked Service credentials or IAM role assignment | | ForEach very slow | Running iterations one-by-one (sequential) instead of parallel | Set **Batch Count** in ForEach settings to run several months concurrently (start

small, e.g., 4, to avoid throttling) |

**Interview line:** *"I used ADF's Monitor tab to debug failed runs at the individual activity level rather than re-running the whole pipeline — this let me isolate a bad URL pattern for one specific month without wasting compute reprocessing months that already succeeded."*

---

## 12. Interview Cheat Sheet

Quick answers you should be able to give fluently after finishing this:

**Q: Walk me through your ingestion pipeline.**

"I built an ADF pipeline that pulls NYC TLC's monthly parquet files from their public HTTP endpoint and lands them untouched into a `raw` zone in ADLS Gen2. It's parameterized — a ForEach activity loops over an array of year-month strings, and both my source and sink datasets use those as parameters to dynamically build the file URL and destination path. This means scaling from 1 month to 5 years of data is a one-line array change."

**Q: Why ADLS Gen2 instead of regular Blob Storage?**

"ADLS Gen2 adds a hierarchical namespace on top of Blob Storage, giving me real folder-level operations and better performance for Spark/Databricks, which is critical since I process this data downstream. It also lets me cleanly implement medallion zones — raw, bronze, silver, gold — as actual folders."

**Q: Why not just write a Python script to download the files?**

"I could have, but ADF gives me managed orchestration for free — retries, monitoring, scheduling, and parameterized looping — without provisioning or maintaining any infrastructure myself. It's also the standard tool in Azure data engineering stacks, so the skills transfer directly."

**Q: How did you secure access between ADF and storage?**

"Started with storage account key auth to move fast, then migrated to Managed Identity with a `Storage Blob Data Contributor` role assignment — so there's no secret key stored or rotated manually."

**Q: What would you change for a production version of this pipeline?**

"I'd add: a tumbling window trigger tied to TLC's actual publish schedule, alerting on failure (via Azure Monitor/Logic Apps), incremental/idempotent loads with a control table tracking which months are already ingested, and a Self-Hosted IR if the source were behind a private network."

---

## 13. Glossary (quick lookup)

| Term | Meaning |
|---|---|
| Resource Group | A folder grouping related Azure resources for management/billing/cleanup |
| ADLS Gen2 | Blob Storage + hierarchical namespace (real folders) + big-data-optimized access |
| Hierarchical Namespace | The setting that turns Blob Storage into a real data lake with folders |
| Container | Top-level "drive" inside a storage account |
| Linked Service | Saved connection info to a source or destination system in ADF |
| Dataset | A pointer to a specific file/table + its format, using a Linked Service |
| Activity | One task/step inside an ADF pipeline (e.g., Copy, ForEach) |
| Pipeline | The full workflow made of one or more Activities |
| Integration Runtime (IR) | The compute engine that actually executes ADF activities |
| ForEach Activity | Loops an inner activity over a list of items |
| Dynamic Content | ADF's expression language (`@...`) for building values at runtime |
| ELT | Extract, Load, Transform — load raw data first, transform later (this project's pattern) |
| SAS Token | A temporary, scoped access credential |
| Managed Identity | An Azure-issued identity for a service, avoiding stored secrets |
| RBAC | Role-Based Access Control — granting permissions via named roles |

---

### Next Doc

Once this is working and you can explain every line above out loud without notes, move on to **Week 2-3: Databricks + Delta Lake + Spark performance tuning** — that's where the real "big data" interview substance comes from. Let me know when you're ready and I'll build that guide the same way.